```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.cluster import KMeans
from sklearn.metrics import r2_score, mean_absolute_error, mean_squa
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

RANDOM_STATE = 42

pd.set_option("display.max_columns", 200)
pd.set_option("display.width", 200)
```

```
!ls -lah
```

```
total 6.8M
drwxr-xr-x 1 root root 4.0K Feb 27 22:23 .
drwxr-xr-x 1 root root 4.0K Feb 27 22:16 ..
-rw-r--r-- 1 root root 6.8M Feb 27 22:23 AB_NYC_2019.csv
drwxr-xr-x 4 root root 4.0K Jan 16 14:24 .config
drwxr-xr-x 1 root root 4.0K Jan 16 14:24 sample_data
```

```
import pandas as pd

df = pd.read_csv("/content/AB_NYC_2019.csv")
df.head()
```

	id	name	host_id	host_name	neighbourhood_group	neig
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	I

```
print("Shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isna().sum().sort_values(ascending=False).head(10))
```

```
Shape: (48895, 16)

Columns:
['id', 'name', 'host_id', 'host_name', 'neighbourhood_group', 'neigh

Data types:
id                                int64
name                             object
host_id                           int64
host_name                        object
neighbourhood_group              object
neighbourhood                    object
latitude                        float64
longitude                       float64
room_type                        object
price                             int64
minimum_nights                    int64
number_of_reviews                 int64
last_review                      object
reviews_per_month               float64
calculated_host_listings_count    int64
availability_365                  int64
dtype: object

Missing values:
last_review            10052
reviews_per_month      10052
host_name                 21
name                      16
neighbourhood_group        0
neighbourhood              0
id                         0
host_id                    0
longitude                  0
latitude                   0
dtype: int64
```

```
df = df.drop(columns=["id", "name", "host_id", "host_name", "last_r
df.head()
```

	neighbourhood_group	neighbourhood	latitude	longitude	room_type
0	Brooklyn	Kensington	40.64749	-73.97237	Private room
1	Manhattan	Midtown	40.75362	-73.98377	Entire home/apt
2	Manhattan	Harlem	40.80902	-73.94190	Private room
3	Brooklyn	Clinton Hill	40.68514	-73.95976	Entire home/apt
4	Manhattan	East Harlem	40.79851	-73.94399	Entire home/apt

```
df["reviews_per_month"] = df["reviews_per_month"].fillna(0)

print(df.isna().sum().sort_values(ascending=False))
```

```
neighbourhood_group               0
neighbourhood                     0
latitude                          0
longitude                         0
room_type                         0
price                             0
minimum_nights                    0
number_of_reviews                 0
reviews_per_month                 0
calculated_host_listings_count    0
availability_365                  0
dtype: int64
```

```
X = df.drop("price", axis=1)
y = df["price"]

print("Features shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features shape: (48895, 10)
Target shape: (48895,)
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=RANDOM_STATE
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (39116, 10)
Test shape: (9779, 10)
```

```
categorical_cols = X.select_dtypes(include=["object"]).columns.toli
numerical_cols = X.select_dtypes(exclude=["object"]).columns.tolist

print("Categorical:", categorical_cols)
print("Numerical:", numerical_cols)
```

```
Categorical: ['neighbourhood_group', 'neighbourhood', 'room_type']
Numerical: ['latitude', 'longitude', 'minimum_nights', 'number_of_re
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Preprocessing for numerical data
numeric_transformer = StandardScaler()

# Preprocessing for categorical data
categorical_transformer = OneHotEncoder(handle_unknown="ignore")

# Combine preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numerical_cols),
        ("cat", categorical_transformer, categorical_cols)
    ]
)
```

```
from sklearn.linear_model import LinearRegression

linear_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LinearRegression())
])

# Train
linear_model.fit(X_train, y_train)

# Predict
y_pred_linear = linear_model.predict(X_test)
```

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squ
import numpy as np

r2 = r2_score(y_test, y_pred_linear)
mae = mean_absolute_error(y_test, y_pred_linear)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_linear))

print("Linear Regression Performance:")
print("R²:", round(r2, 4))
print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
```

```
Linear Regression Performance:
R²: 0.1359
MAE: 68.15
RMSE: 195.52
```

```
from sklearn.linear_model import Ridge

ridge_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", Ridge(alpha=1.0))
])

ridge_model.fit(X_train, y_train)

y_pred_ridge = ridge_model.predict(X_test)

r2_ridge = r2_score(y_test, y_pred_ridge)
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
rmse_ridge = np.sqrt(mean_squared_error(y_test, y_pred_ridge))

print("Ridge Regression Performance:")
print("R²:", round(r2_ridge, 4))
print("MAE:", round(mae_ridge, 2))
print("RMSE:", round(rmse_ridge, 2))
```

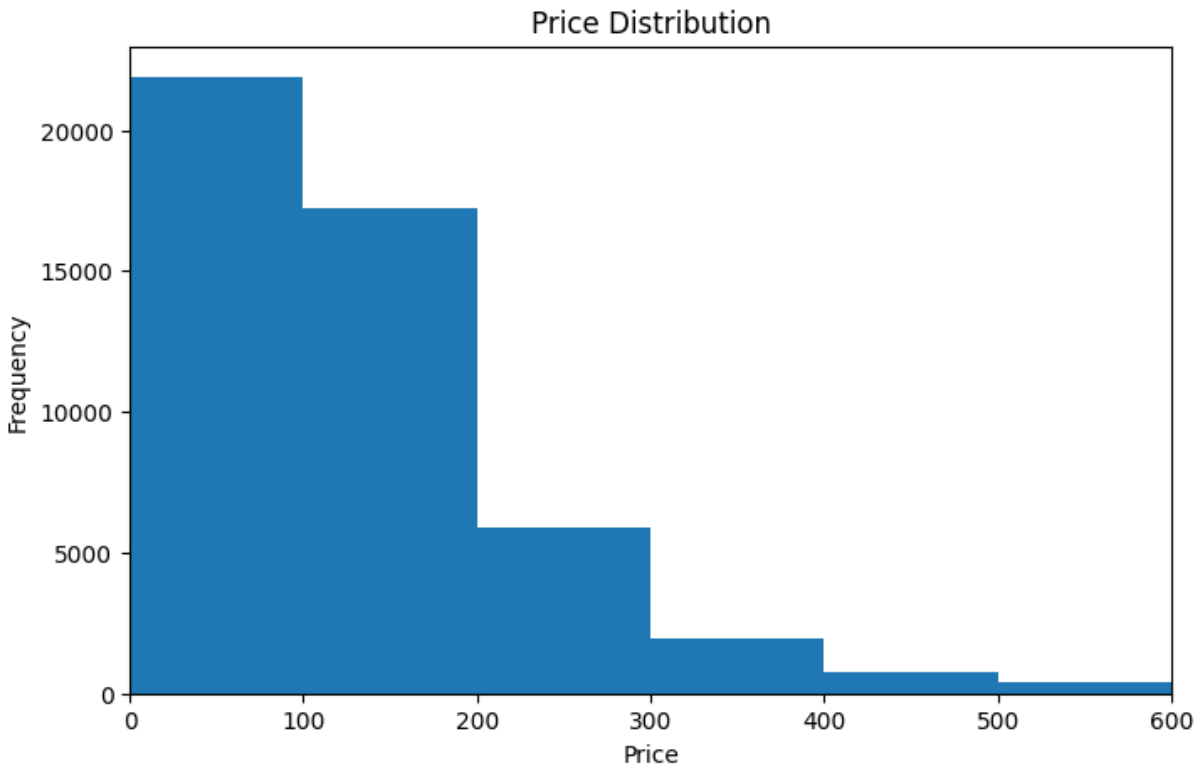
```
Ridge Regression Performance:
R²: 0.137
MAE: 68.07
RMSE: 195.39
```

```
# Get feature names after preprocessing
feature_names = (
    numerical_cols +
```

```
    list(ridge_model.named_steps["preprocessor"]
         .named_transformers_["cat"]
         .get_feature_names_out(categorical_cols))
)

coefficients = ridge_model.named_steps["model"].coef_

import pandas as pd

coef_df = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients
})

coef_df = coef_df.sort_values(by="Coefficient", key=abs, ascending=

coef_df.head(15)
```

	Feature	Coefficient
186	neighbourhood_Sea Gate	447.712904
173	neighbourhood_Randall Manor	300.649646
94	neighbourhood_Fort Wadsworth	298.599622
179	neighbourhood_Riverdale	290.755001
209	neighbourhood_Tribeca	243.629914
169	neighbourhood_Prince's Bay	168.354075
18	neighbourhood_Battery Park City	145.735515
11	neighbourhood_group_Staten Island	-140.203413
48	neighbourhood_City Island	137.927490
87	neighbourhood_Flatiron District	118.202082
83	neighbourhood_Far Rockaway	107.557121
93	neighbourhood_Fort Hamilton	-95.202766
113	neighbourhood_Huguenot	-93.817674
17	neighbourhood_Bath Beach	-89.014954
33	neighbourhood_Breezy Point	79.933808

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.hist(df["price"], bins=100)
plt.title("Price Distribution")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.xlim(0, 600)
plt.show()
```



```
import numpy as np

y_log = np.log1p(y)

X_train, X_test, y_train_log, y_test_log = train_test_split(
    X, y_log,
    test_size=0.2,
    random_state=RANDOM_STATE
)

linear_model_log = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LinearRegression())
])

linear_model_log.fit(X_train, y_train_log)

y_pred_log = linear_model_log.predict(X_test)

r2_log = r2_score(y_test_log, y_pred_log)
mae_log = mean_absolute_error(y_test_log, y_pred_log)
rmse_log = np.sqrt(mean_squared_error(y_test_log, y_pred_log))

print("Log-Linear Regression Performance:")
print("R²:", round(r2_log, 4))
print("MAE:", round(mae_log, 4))
print("RMSE:", round(rmse_log, 4))
```

```
Log-Linear Regression Performance:
R²: 0.5499
MAE: 0.3347
RMSE: 0.4615
```

```
from sklearn.linear_model import Ridge

ridge_model_log = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", Ridge(alpha=1.0))
])

ridge_model_log.fit(X_train, y_train_log)

y_pred_ridge_log = ridge_model_log.predict(X_test)

r2_ridge_log = r2_score(y_test_log, y_pred_ridge_log)
mae_ridge_log = mean_absolute_error(y_test_log, y_pred_ridge_log)
rmse_ridge_log = np.sqrt(mean_squared_error(y_test_log, y_pred_ridg

print("Log-Ridge Regression Performance:")
print("R²:", round(r2_ridge_log, 4))
print("MAE:", round(mae_ridge_log, 4))
print("RMSE:", round(rmse_ridge_log, 4))
```

```
Log-Ridge Regression Performance:
R²: 0.5503
MAE: 0.3346
RMSE: 0.4613
```

```
from sklearn.ensemble import RandomForestRegressor

rf_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RandomForestRegressor(
        n_estimators=80,       # slightly fewer trees
        max_depth=15,          # limit depth
        min_samples_split=10,
        random_state=RANDOM_STATE,
        n_jobs=-1
    ))
])

rf_model.fit(X_train, y_train_log)

y_pred_rf = rf_model.predict(X_test)

r2_rf = r2_score(y_test_log, y_pred_rf)
mae_rf = mean_absolute_error(y_test_log, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test_log, y_pred_rf))

print("Random Forest (Log Target) Performance:")
print("R²:", round(r2_rf, 4))
print("MAE:", round(mae_rf, 4))
print("RMSE:", round(rmse_rf, 4))
```

```
Random Forest (Log Target) Performance:
R²: 0.6153
MAE: 0.3036
RMSE: 0.4266
```

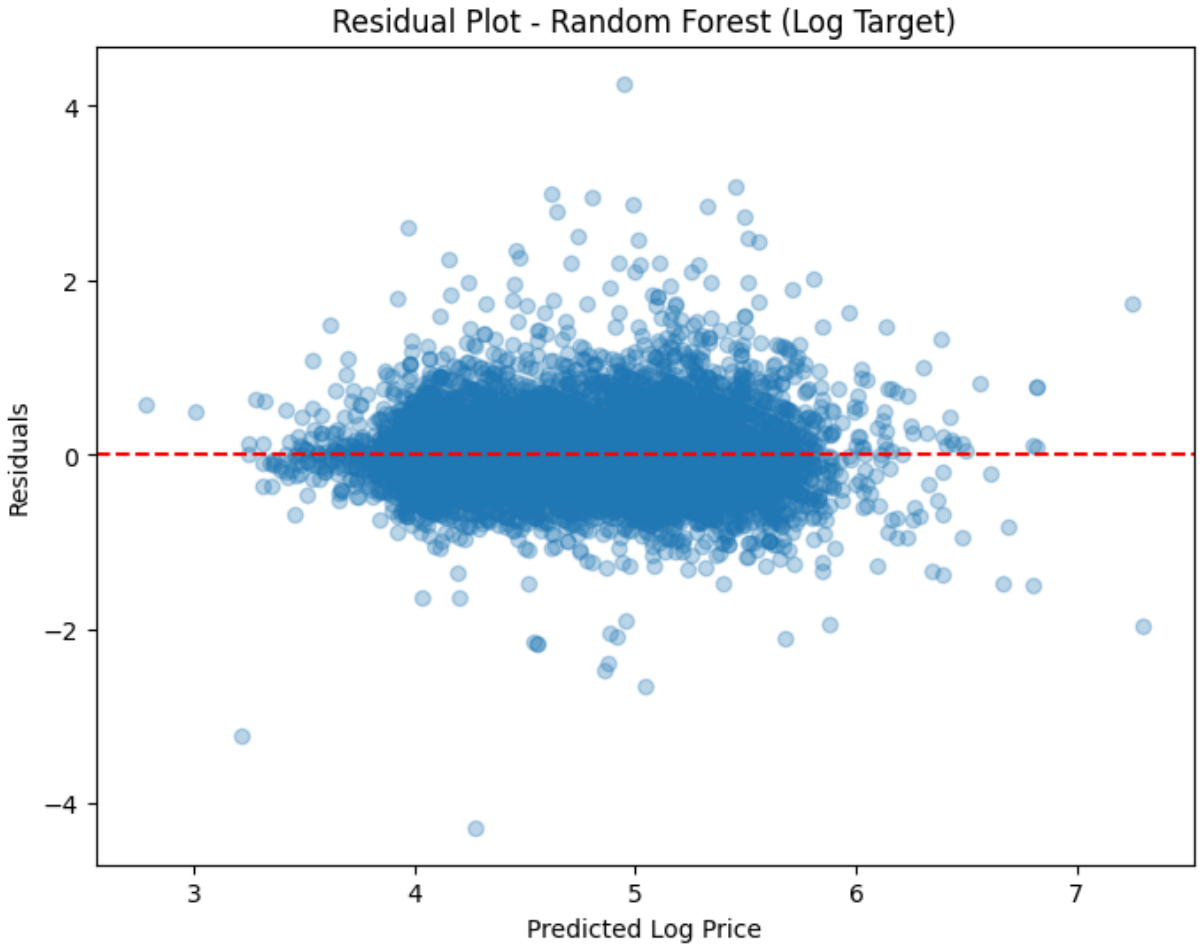
```
import pandas as pd

rf = rf_model.named_steps["model"]

feature_names = rf_model.named_steps["preprocessor"].get_feature_na

importances = rf.feature_importances_

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

importance_df.head(15)
```

	Feature	Importance
232	cat__room_type_Entire home/apt	0.482046
1	num__longitude	0.117665
0	num__latitude	0.109782
6	num__availability_365	0.063645
2	num__minimum_nights	0.042117
9	cat__neighbourhood_group_Manhattan	0.040624
4	num__reviews_per_month	0.032090
5	num__calculated_host_listings_count	0.028549
3	num__number_of_reviews	0.025384
139	cat__neighbourhood_Midtown	0.006986
233	cat__room_type_Private room	0.006594
234	cat__room_type_Shared room	0.003681
209	cat__neighbourhood_Tribeca	0.002299
25	cat__neighbourhood_Bedford-Stuyvesant	0.002233
179	cat__neighbourhood_Riverdale	0.001550

```
import pandas as pd

results_df = pd.DataFrame({
    "Model": [
        "Linear (Raw)",
        "Linear (Log)",
        "Ridge (Log)",
        "Random Forest (Log)"
    ],
    "R2": [
        0.1359,
        r2_log,
        r2_ridge_log,
        r2_rf
    ],
    "MAE": [
        68.15,
        mae_log,
        mae_ridge_log,
        mae_rf
    ],
    "RMSE": [
        195.52,
        rmse_log,
        rmse_ridge_log,
        rmse_rf
    ]
})

results_df
```

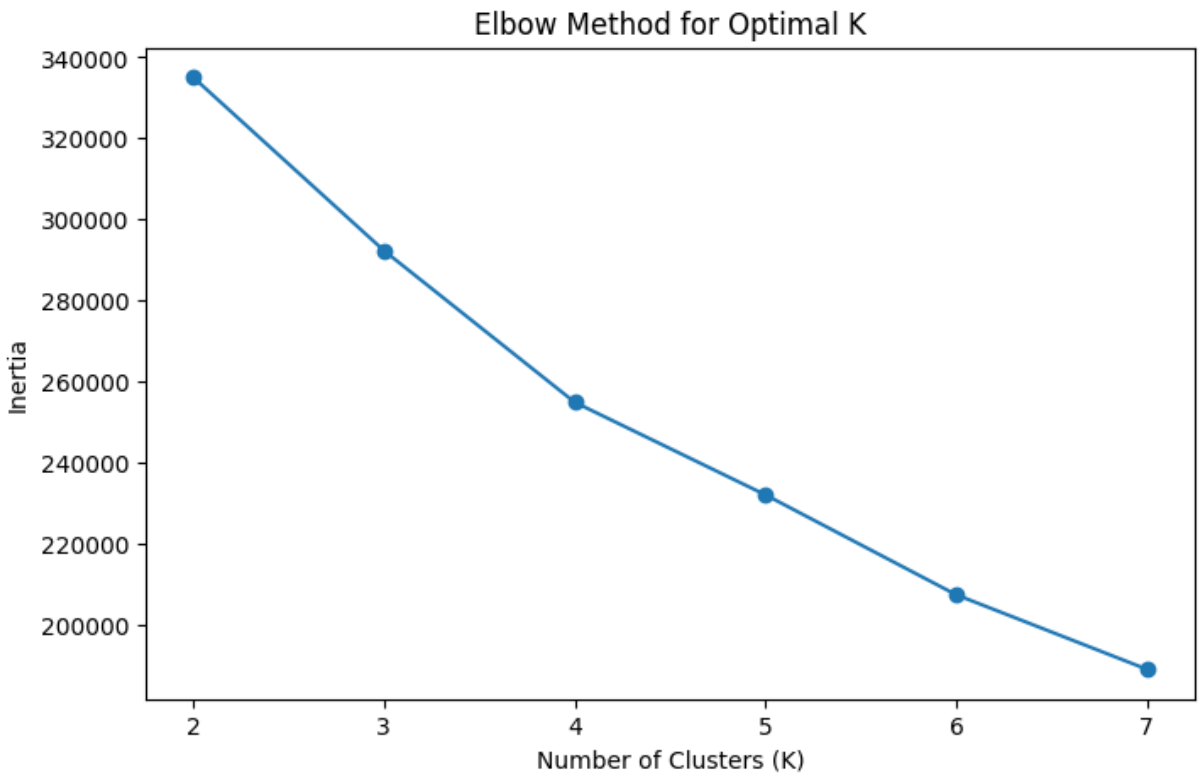
	Model	R2	MAE	RMSE
0	Linear (Raw)	0.135900	68.150000	195.520000
1	Linear (Log)	0.549855	0.334720	0.461474
2	Ridge (Log)	0.550285	0.334622	0.461253
3	Random Forest (Log)	0.615308	0.303574	0.426606

```
import matplotlib.pyplot as plt
import numpy as np

residuals = y_test_log - y_pred_rf

plt.figure(figsize=(8,6))
plt.scatter(y_pred_rf, residuals, alpha=0.3)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel("Predicted Log Price")
plt.ylabel("Residuals")
plt.title("Residual Plot - Random Forest (Log Target)")
plt.show()
```



```
from sklearn.preprocessing import StandardScaler

# Select features for clustering
cluster_features = df[[
    "price",
    "availability_365",
    "number_of_reviews",
    "reviews_per_month",
    "minimum_nights",
    "calculated_host_listings_count",
    "latitude",
    "longitude"
]].copy()

# Log-transform price (consistent with regression insight)
cluster_features["price"] = np.log1p(cluster_features["price"])

# Scale features
scaler = StandardScaler()
cluster_scaled = scaler.fit_transform(cluster_features)
```

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertia = []

K_range = range(2, 8)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init
    kmeans.fit(cluster_scaled)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8,5))
plt.plot(K_range, inertia, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method for Optimal K")
plt.show()
```



```
kmeans_final = KMeans(n_clusters=4, random_state=RANDOM_STATE, n_in
df["Cluster"] = kmeans_final.fit_predict(cluster_scaled)

df["Cluster"].value_counts()
```

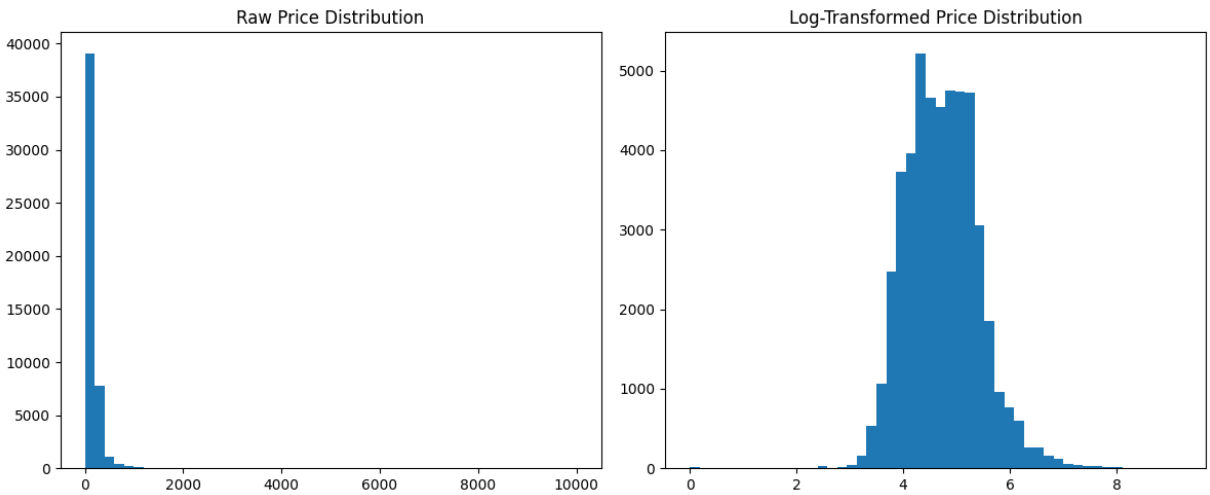
Cluster	count
0	28475
2	12277
1	7584
3	559

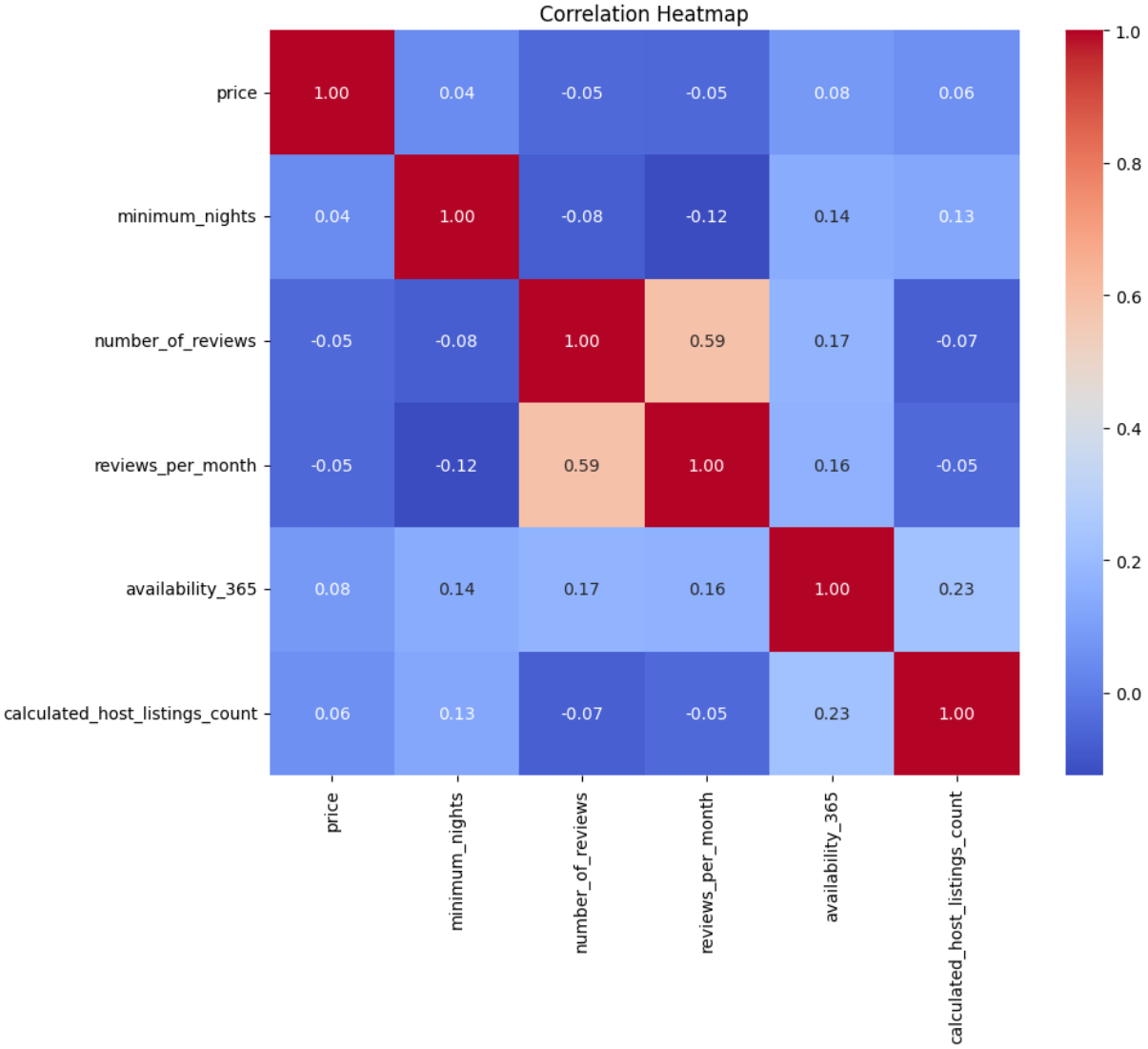
dtype: int64

```
cluster_summary = df.groupby("Cluster")[[
    "price",
    "availability_365",
    "number_of_reviews",
    "reviews_per_month",
    "minimum_nights",
    "calculated_host_listings_count"
]].mean()

cluster_summary
```

Cluster	price	availability_365	number_of_reviews	reviews_per_
0	138.523512	22.821001	8.430026	0.
1	120.749736	153.863660	96.267537	3.
2	199.880182	288.363688	13.566588	0.
3	273.928444	281.703041	2.343470	0.

```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.hist(df["price"], bins=50)
plt.title("Raw Price Distribution")

plt.subplot(1,2,2)
plt.hist(np.log1p(df["price"]), bins=50)
plt.title("Log-Transformed Price Distribution")

plt.tight_layout()
plt.show()
```




```
import seaborn as sns

plt.figure(figsize=(10,8))
corr = df[[
    "price",
    "minimum_nights",
    "number_of_reviews",
```

```
    "reviews_per_month",
    "availability_365",
    "calculated_host_listings_count"
]].corr()

sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```